

The F2Z polynomial commitment scheme

Relations, reductions, and a normative prover–verifier specification

Normative v1 profile. This document specifies the byte-exact direct chunked mod- q , recursive-opening F2Z core and the required black-box seams to registered application, GKR, and constrained-code profiles. It begins with an application PIMSAT statement, commits to the bit-decomposition of its witness, and runs an application-supplied PIOP whose output has the weighted coefficient shape required by the PCS core. A virtual-witness extension additionally supplies and verifies its partial-core predecessor, then applies the adjoint and a compatible ring-switch adapter. Any still-open entry in Section 6 rejects the affected extension; it does not weaken the v1 wire rules of Section 5.

1 Instance, parameters, and constraints

1.1 Typed objects and algebraic domains

An execution is determined by public parameters, an application statement, a derived PCS statement, a commitment, and local prover state:

$$\begin{aligned} \text{pp} &:= (\text{domains}, \varphi, \psi, \mathcal{C}, \text{hash}, \text{transcript}, \text{profile}), \\ x_{\text{app}} &:= (\mathbf{x}, \mathbf{p}, \mathbf{I}, (t, s, W), g), & x_{\text{core}} &:= ((t, s, W), \text{claim}, g), \\ \text{com} &:= (\rho), \\ \text{st}_P &:= (\mathbf{f}, D, \mathbf{b}, B_{\text{bit}}, P, O, \text{tree}). \end{aligned} \tag{1}$$

Here $(\mathbf{x}, \mathbf{p}, \mathbf{I})$ specifies the public PIMSAT instance of Section 2. The application PIOP derives **claim** and hence x_{core} ; neither is trusted as an independent prover input. The commitment is public before the first PIOP challenge, and st_P is never transmitted.

Abstractly, R_M and R are commutative unital rings, M is a finite free R_M -module, and N is an R -module. Write $\iota_A : \mathbb{Z} \rightarrow A$ for the canonical integer map of any unital ring A ; in particular, $\iota_R(b) = b1_R$ for $b \in \{0, 1\}$. Whenever N is scalarized, it is finite free with a fixed basis, or the profile supplies a separating family of R -linear functionals. A public unital ring homomorphism φ and a φ -semilinear public map ψ satisfy

$$\varphi : R_M \rightarrow R, \quad \psi : M \rightarrow N, \quad \psi(ax + by) = \varphi(a)\psi(x) + \varphi(b)\psi(y)$$

connect the witness and evaluation domains. Packing uses $\mathbb{F}_{\text{bin}} \cong \mathbb{F}_{2^\kappa}$ with an ordered $\mathbb{F}_2$ -basis $(\beta_v)_{v < \kappa}$ and a linear code $\mathcal{C}$ over $\mathbb{F}_{\text{bin}}$; any padding to a multiple of κ is canonical and statement-bound.

The concrete mod- q profile fixes

$$R = \mathbb{F}_q, \quad q = 2^{100} - 15, \quad \mathbb{F}_{\text{bin}} = \mathbb{K} = \mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1) \cong \mathbb{F}_{2^{128}}. \tag{2}$$

Write $\pi_2 : \mathbb{Z} \rightarrow \mathbb{F}_2$, $\pi_q : \mathbb{Z} \rightarrow \mathbb{F}_q$, and $\text{can}_q : \mathbb{F}_q \rightarrow [0, q]$. Linear evaluations are in $\mathbb{F}_q$; packing, product reductions, and commitment openings are in $\mathbb{K}$. Its wire identifier, canonical encodings, transcript and Merkle algorithms, recursive-opening configuration, and PoW schedule are fixed by Section 5; they are not runtime-selected proof fields. The exact constrained-code transform that produces the committed matrices is selected by the statement-bound `code_profile_id` and is unsupported unless that identifier resolves to a published canonical construction.

1.2 Tensor witness, packing, and commitment

Every entry is normative; only the shape/interface, fixed-scalar, and generator requirements are pre-transcript checks.

Object	Exact definition	Type and rule
Shape and cells	$\mathbf{b} := \text{bits}(\mathbf{f})$, $W = 2^w$, and $D(b, c) \in [0, 2^W)$, $d := t + w$, $m := d + s$, $m_p := m - 7$.	(t, s, W) is public, $\mathbf{f}$ is bounded, the bit-decomposition is canonical, and $(b, c) \in \{0, 1\}^t \times \{0, 1\}^s$.
Bit tensor (3)	$B_{\text{bit}}[c][\lfloor (b \ll w) \rfloor j] := \text{bit}_j(D(b, c)),$ $\text{INT}(D(b, c)) := \sum_{j=0}^{W-1} 2^j B_{\text{bit}}[c][\lfloor (b \ll w) \rfloor j].$	$B_{\text{bit}} \in \mathbb{F}_2^{2^s \times 2^d}$, $0 \leq j < W$, and $\mathbf{b} = \text{vec}(B_{\text{bit}})$ has 2^m bits.
Pack and commit (4)	$P[c2^{d-7} + i_{\text{hi}}] := \sum_{v=0}^{127} B_{\text{bit}}[c][\lfloor (i_{\text{hi}} \ll 7) \rfloor v] \beta_v,$ $O := \text{Enc}_{\mathcal{C}}(P), \quad \rho := \text{Root}(O).$	$0 \leq i_{\text{hi}} < 2^{d-7}$, $0 \leq v < 128$, and $P \in \mathbb{K}^{2^m p}$. The seven low index bits select the 128 coefficients of the fixed ordered $\mathbb{F}_2$ -basis of $\mathbb{K}$.

1.3 Weighted and MLE successor forms

Weighted claim	$\text{claim}_{\text{wt}} = (\mathbf{w}, \mathbf{w}', y), \quad \mathbf{w} = 2^t, \quad \mathbf{w}' = 2^s.$	$0 \leq w_b < 2^{q\text{bits}}$ is an integer; $w'_c, y \in \mathbb{F}_q$.
Target equality (5)	$y = \sum_{c \in \{0, 1\}^s} w'_c \pi_q \left(\sum_{b \in \{0, 1\}^t} w_b \text{INT}(D(b, c)) \right).$	The equality is in $\mathbb{F}_q$.
Bit coefficient (6)	$u_{c, b, j} := \pi_q(w_b 2^j) w'_c, \quad i := \lfloor (b \ll w) \rfloor j.$	$0 \leq i < 2^d$ and $u_{c, b, j} \in \mathbb{F}_q$. This coefficient is derived, not supplied.
MLE specialization (7)	If the application PIOP exposes transcript-derived $r_1 \in \mathbb{F}_q^t$ and $r_2 \in \mathbb{F}_q^s$, its successor may use $w_b := \text{can}_q(\text{eq}(b, r_1)), \quad w'_c := \text{eq}(c, r_2),$ $y := \pi_q \circ \text{INT}(D)(r_1, r_2).$	The verifier derives these weights and the bit coefficients above; the core still receives $(\mathbf{w}, \mathbf{w}', y)$.

1.4 Chunking and admissibility

Chunk decomposition (8)	Parameters. $q_{\text{bits}} := 100$, $c_w := 127 - t - W$, $L := \max(\lceil q_{\text{bits}}/c_w \rceil, 1).$	$0 \leq \ell < L$; the admissibility rows below imply $c_w > 0$.
	Digits. $\omega_b^{(\ell)} := \lfloor w_b / 2^{c_w \ell} \rfloor \bmod 2^{c_w}, \quad w_b = \sum_{\ell=0}^{L-1} 2^{c_w \ell} \omega_b^{(\ell)}.$	$0 \leq \omega_b^{(\ell)} < 2^{c_w}.$
	Columns. $u_c^{(\ell)} := \sum_b \omega_b^{(\ell)} \text{INT}(D(b, c)).$	$0 \leq u_c^{(\ell)} < 2^{c_w + t + W} = 2^{127}.$

Object	Exact definition	Type and rule
Active set (9)	$\mathcal{A} := \{\ell < L : \exists b, \omega_b^{(\ell)} \neq 0\}.$	$\mathcal{A}$ is public and derived only from w . Transmit only active chunks, in increasing order; set $u_c^{(\ell)} := 0$ for $\ell \notin \mathcal{A}$; require $\mathcal{A} \neq \emptyset$.
Shape and interface (10)	$W \in 2^{\mathbb{N}}, \quad d \geq 7, \quad t + W \leq 126, \quad 22 \leq m \leq 35.$	Require every implied dimension, canonical public values, and the correct claim variant.
Fixed scalars	$\theta_2, \theta_3 \in \mathbb{K} \setminus \{0, 1\}, \quad \theta_2 \neq \theta_3.$	These values are fixed by the selected profile.
Generator (11)	$g \in \mathbb{K}^\times, \quad \text{ord}(g) = 2^{128} - 1,$ $\text{ord}(g) = 2^{128} - 1 \iff g^{(2^{128}-1)/p} \neq 1 \quad \text{for every prime } p \mid 2^{128} - 1.$	Together with $u_c^{(\ell)} < 2^{127}$, full order makes $u \mapsto g^u$ injective over every accepted fold.

2 Relations, reductions, and proof objects

2.1 How to read this section

A paper-level encoded relation contains triples $(x, O; w)$: x is the explicit instance, O is the implicit encoded oracle, and w is the witness. For a public witness map T , the binding condition is

$$O = \text{Enc}_C(T(w)).$$

Merkle compilation commits to O with the public root $\rho := \text{Root}(O)$; ρ is not a proof field. Every relation below is additionally parameterized by the fixed public context $\chi := (\text{pp}, t, s, W, g)$ and its derived dimensions. The context is omitted from displayed instances and is carried unchanged by every edge. The edge notation is

$$\mathcal{R}_0 \equiv_F \mathcal{R}_1 \quad \text{deterministic retyping}, \quad \mathcal{R}_0 \xRightarrow{\Pi} \mathcal{R}_1 \quad \text{interactive reduction},$$

$$\mathcal{R}_0 \dashrightarrow \mathcal{R}_1 \quad \text{conditional extension or classification only.}$$

Each subsection defines one source relation and lists its direct outgoing reductions. A proof fragment lists transmitted values only. Challenges, successor coefficients, exponentiated values, batching coefficients, and work tables are verifier-derived.

2.2 The application PIMSAT relations

Let $S := R_M[X]$, and let $M \subseteq S$ be a statement-bound finite free R_M -submodule. Fix a public instance $\mathbf{x} \in S^{\ell_x}$, a bounded witness domain $\mathcal{W}_B \subseteq M^r$, ideals $I_a \triangleleft S$, and constraint polynomials $p_j^{(a)} \in S[X, Y]$. The paper first uses the unencoded relation

$$\mathcal{R}_{\text{PIMSAT}}^{\text{app}} := \left\{ (\mathbf{x}, \perp; \mathbf{f}) : \mathbf{f} \in \mathcal{W}_B, \quad p_j^{(a)}(\mathbf{x}, \mathbf{f}) \in I_a \quad (\forall a, j) \right\}. \quad (12)$$

Encoding the witness gives the encoded application relation

$$\mathcal{R}_{\text{PIMSAT}}^{\text{enc}} := \left\{ (\mathbf{x}, O; \mathbf{f}) : \mathbf{f} \in \mathcal{W}_B, \quad O = \text{Enc}_C(\text{Pack}(\pi_2(\text{bits}(\mathbf{f})))), \right. \\ \left. p_j^{(a)}(\mathbf{x}, \mathbf{f}) \in I_a \quad (\forall a, j) \right\}. \quad (13)$$

Direct reductions.

- **Reduction 0: Compile the witness oracle.**

$$\mathcal{R}_{\text{PIMSAT}}^{\text{app}} \xRightarrow{\text{Encode}} \mathcal{R}_{\text{PIMSAT}}^{\text{enc}}.$$

The IOP supplies the encoded oracle O ; Merkle compilation exposes $\rho = \text{Root}(O)$. The constraint predicates are unchanged.

- **Reduction 1: Reduce ideal membership to application PESAT.**

$$\mathcal{R}_{\text{PIMSAT}}^{\text{enc}} \xRightarrow{\Pi_{\text{app},1}} \mathcal{R}_{\text{PESAT}}^{\text{app}}.$$

- **Proof:** $\pi_{\text{app},1}$ carries the PIOP messages that turn ideal-membership constraints into field equations.
- **Result:** the verifier derives K_{app} , ψ_{app} , $\hat{\mathbf{x}}$, and $\hat{\mathbf{p}}$; O is unchanged.
- **Scope:** application-supplied, before the F2Z PCS core.
- **Classification: Equality and congruence constraints.** $I_a = (0)$ gives equality; nonzero I_a gives congruence or quotient-ring constraints. Both use the same reduction.

2.3 The encoded application PESAT relation

Fix the verifier-derived application field K_{app} , a public map $\psi_{\text{app}} : M \rightarrow K_{\text{app}}$, and projected constraint polynomials $\hat{p}_i \in K_{\text{app}}[X, Y]$. The encoded application PESAT relation retains the original bounded witness:

$$\mathcal{R}_{\text{PESAT}}^{\text{app}} := \left\{ ((\hat{\mathbf{x}}, \hat{\mathbf{p}}), O; \mathbf{f}) : \mathbf{f} \in \mathcal{W}_B, \quad O = \text{Enc}_C(\text{Pack}(\pi_2(\text{bits}(\mathbf{f})))), \right. \\ \left. \hat{p}_i(\hat{\mathbf{x}}, \psi_{\text{app}}(\mathbf{f})) = 0 \quad (\forall i) \right\}. \quad (14)$$

Direct reductions.

- **Reduction 1: Linearize the application PESAT claim.**

$$\mathcal{R}_{\text{PESAT}}^{\text{app}} \xRightarrow{\Pi_{\text{app},2}} \mathcal{R}_{\text{Lin-ModFieldBin}}.$$

- **Proof:** $\pi_{\text{app},2}$ carries the application reduction from the projected constraints to one scalar linear claim.
- **Result:** the verifier derives $(\mathbf{v}, \mu)$ and ψ ; the same witness and O pass to $\mathcal{R}_{\text{Lin-ModFieldBin}}$.
- **Scope:** application-supplied. Together, $\pi_{\text{PIOP}} := (\pi_{\text{app},1}, \pi_{\text{app},2})$ is the complete application proof fragment.
- **Classification: A single linear constraint is LMFB.** If $\hat{\mathbf{p}}$ is one linear polynomial, the relation is already Lin-ModFieldBin; no extra message is needed.

2.4 The Lin-ModFieldBin relation

Let $(\gamma_j)_{j < D_M}$ be the fixed basis of M , and assume a statement-bound bijection $\text{bits} : \mathcal{W}_B \leftrightarrow \{0, 1\}^n$ with fixed reconstruction:

$$f_i = \sum_{k < B} \sum_{j < D_M} b_{i,k,j} 2^k \gamma_j, \quad b_{i,k,j} \in \{0, 1\}, \quad \mathbf{b} = \text{bits}(\mathbf{f}).$$

At this seam a higher-rank target has already been expanded into scalar coordinates; write $\psi : M \rightarrow R$ for the resulting φ -semilinear map.

The module relation is

$$\mathcal{R}_{\text{Lin-ModFieldBin}} := \left\{ ((v, \mu), O; f) : f \in \mathcal{W}_B, \quad v \in R^r, \quad \mu \in R, \quad O = \text{Enc}_C(\text{Pack}(\pi_2(\text{bits}(f)))) \right. \\ \left. \sum_i v_i \psi(f_i) = \mu \quad \text{in } R \right\}. \quad (15)$$

Direct reductions.

- **Reduction 1: Deterministic Bitify.**

$$\mathcal{R}_{\text{Lin-ModFieldBin}} \equiv \text{Bitify } \mathcal{R}_{\text{Lin-BitsDual}}, \quad u_{i,k,j} := v_i \varphi(\iota_{R_M}(2^k)) \psi(\gamma_j).$$

The witness is retyped as $\mathbf{b} := \text{bits}(f)$; the same encoded oracle O remains bound to it. Bitify transmits nothing and performs no new Booleanity test.

- **Required preprocessing for a higher-rank target.** If N has rank greater than one, first expand it into scalar LMFB claims. Then bitify each coordinate or use an application-defined batch; no canonical batching proof is specified here.

2.5 The Lin-BitsDual relation

The core bit relation is

$$\mathcal{R}_{\text{Lin-BitsDual}} := \left\{ ((u, \mu), O; \mathbf{b}) : \mathbf{b} \in \{0, 1\}^n, \quad u \in R^n, \quad \mu \in R, \quad O = \text{Enc}_C(\text{Pack}(\pi_2(\mathbf{b}))), \right. \\ \left. \langle \iota_R(\mathbf{b}), u \rangle_R = \mu \right\}. \quad (16)$$

Suppose the public coefficient has the paper's tensor form $u = u^{(1)} \otimes u^{(2)}$, and reshape the committed bits as $B_{\text{bit}}[c, i]$. With canonical integer representatives γ_i of $u_i^{(1)}$, the prover exposes column values

$$\mu_c := \sum_i \gamma_i B_{\text{bit}}[c, i] \in \mathbb{Z}.$$

The verifier immediately checks the tensor reconstruction

$$\sum_c u_c^{(2)} \iota_R(\mu_c) = \mu. \quad (17)$$

It then derives $y_i := g^{\gamma_i}$ and $\nu_c := g^{\mu_c}$, obtaining

$$\prod_i (1 + (y_i - 1) B_{\text{bit}}[c, i]) = \nu_c \quad (\forall c).$$

Direct reductions.

- **Reduction 1: Produce the F2Z encoded PESAT claim.**

$$\mathcal{R}_{\text{Lin-BitsDual}} \xRightarrow{\Pi_1} \mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2^{\mathbb{Z}}}, \mathcal{C}, \text{Pack}).$$

- **Proof:** the paper sends $\pi_1 = (\mu_c)_c$. The verifier checks Equation (17) before accepting the successor.
- **Result:** the verifier derives $y = (g^{\gamma_i})_i$, $\nu = (g^{\mu_c})_c$, and the PESAT instance; the bit witness and O are unchanged.
- **Scope:** the first core F2Z reduction.

- **Profile restriction: weighted coefficient shape.** The chunked mod- q profile requires

$$u_{c,b,j} = \pi_q(w_b 2^j) w'_c, \quad 0 \leq w_b < 2^{q_{\text{bits}}}, \quad w'_c \in \mathbb{F}_q.$$

This restricts the public coefficient; it does not define another relation.

2.5.1 Active-chunk instantiation of Π_1

For the admitted weighted shape, write

$$w_b = \sum_{\ell=0}^{L-1} 2^{c w_\ell} \omega_b^{(\ell)}, \quad u_c^{(\ell)} := \sum_b \omega_b^{(\ell)} \text{INT}(D(b, c)).$$

The proof fragment is

$$\pi_1^{\text{chunk}} := (\nu^{(\ell)}, u^{(\ell)})_{\ell \in \mathcal{A}^\uparrow}, \quad \begin{aligned} u^{(\ell)} &= (u_c^{(\ell)})_{c \in \{0,1\}^s}, \\ \nu^{(\ell)} &= (g^{u_c^{(\ell)}})_{c \in \{0,1\}^s}. \end{aligned} \quad (18)$$

For each active chunk in increasing ℓ , $\nu^{(\ell)}$ is a NARG record and $u^{(\ell)}$ is a checked, non-absorbed hint. Inactive chunks have the derived value $u^{(\ell)} = 0$. The verifier rejects a missing, duplicate, out-of-order, or noncanonical active pair; requires $0 \leq u_c^{(\ell)} < 2^{127}$; and checks $g^{u_c^{(\ell)}} = \nu_c^{(\ell)}$ before its next squeeze. Because $\text{ord}(g) = 2^{128} - 1$, the stated range makes the accepted exponent unique. The checked values reconstruct the paper's column messages:

$$\mu_c := \sum_{\ell=0}^{L-1} 2^{c w_\ell} u_c^{(\ell)} = \sum_b w_b \text{INT}(D(b, c)). \quad (19)$$

Inside Π_1 , immediately after receiving the final active vector, the verifier checks

$$\sum_c w'_c \pi_q(\mu_c) = y. \quad (20)$$

This is the tensor-reconstruction check of Equation (17); it creates no named successor relation and leaves no public scalar obligation for the opening phase. For each active ℓ , the verifier also derives

$$i = (b \ll w) \mid j, \quad \omega_i^{(\ell)} = \omega_b^{(\ell)} 2^j, \quad y_i^{(\ell)} = g^{\omega_i^{(\ell)}}.$$

Thus the single paper target is realized by the profile family

$$\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2^{\mathbb{Z}}}^{(\ell)}, \mathcal{C}, \text{Pack}). \quad (21)$$

2.6 The encoded F2Z PESAT relation

For one active profile chunk, define

$$Q_c^{(\ell)}(B_{\text{bit}}) := \prod_i (1 + (y_i^{(\ell)} - 1) B_{\text{bit}}[c, i]) - \nu_c^{(\ell)}.$$

The corresponding named paper relation is the encoded PESAT instance

$$\mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2^{\mathbb{Z}}}^{(\ell)}, \mathcal{C}, \text{Pack}) \\ := \left\{ ((y^{(\ell)}, \nu^{(\ell)}), O; B_{\text{bit}}) : B_{\text{bit}} \in \mathbb{F}_2^{2^s \times 2^d}, \quad O = \text{Enc}_C(\text{Pack}(B_{\text{bit}})), \quad Q_c^{(\ell)}(B_{\text{bit}}) = 0 \quad (\forall c) \right\}. \quad (22)$$

Omitting ℓ gives the paper's unchunked notation $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2^{\mathbb{Z}}}, \mathcal{C}, \text{Pack})$. The profile's “grand product” is this PESAT relation, not a distinct formal relation.

Direct reductions and classifications.

• Reduction 1: Linearize with GKR.

$$\mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2\mathbb{Z}}, \mathcal{C}, \text{Pack}) \xrightarrow{\Pi_{\text{red}}} \mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}).$$

For the chunked family, Π_{red} is applied once per $\ell \in \mathcal{A}$.

- **Input:** the chunk's PESAT instance, the unchanged root $\rho = \text{Root}(O)$, and the transcript after $\zeta^{(\ell)}$ and $e_0^{(\ell)}$; call this bundle $\mathcal{I}_{\text{GKR}}^{(\ell)}$.
- **Proof:**

$$(\text{tr}', pt_\ell, \mu_\ell) \leftarrow \text{GKR.Verify}(\text{pp}_{\text{GKR}}, \mathcal{I}_{\text{GKR}}^{(\ell)}, \pi_{\text{red}}^{(\ell)}; \text{tr}).$$

$\pi_{\text{red}}^{(\ell)}$ is one canonical GKR fragment. Its profile fixes the messages, challenge order, encodings, tags, and rejection rules; the call consumes the whole fragment and returns tr' .

- **Result:** the verifier derives $(\mathbf{a}_\ell, \mu_\ell)$ for $\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack})$, retaining the same O and bit witness. In the current profile, with $B_{\text{bit}, \mathbb{K}} := \iota_{2 \rightarrow \mathbb{K}}(B_{\text{bit}})$ coordinatewise,

$$\mathbf{a}_\ell[z] = \text{eq}(z, pt_\ell), \quad \widetilde{B_{\text{bit}, \mathbb{K}}}(pt_\ell) = \mu_\ell.$$

This equation defines the LIN successor; it is not another paper relation.

- **Scope:** this edge is solid once the GKR profile fixes a byte-exact transcript and proves the reduction. Its error is $\varepsilon_{\text{red}}^{(\ell)}$ per active chunk.
- **Classification: Degree-zero, zero-ideal PIMSAT.** With $S = \mathbb{K}[X]$, $S_{\leq 0} = \mathbb{K}$, and $I = (0)$, membership means exactly $Q_c^{(\ell)} = 0$. Boolean witnesses keep their bit domain; unrestricted $\mathbb{K}$ variables also require $Z^2 - Z = 0$.

2.7 The encoded LIN relation

Let $\iota_{2 \rightarrow \mathbb{K}} : \mathbb{F}_2 \hookrightarrow \mathbb{K}$ act coordinatewise and put $\mathbf{b}_{\mathbb{K}} := \iota_{2 \rightarrow \mathbb{K}}(\pi_2(\mathbf{b}))$. The paper's linear target is

$$\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}) := \left\{ ((\mathbf{a}, \mu), O; \mathbf{b}) : \mathbf{b} \in \{0, 1\}^n, \quad \mathbf{a} \in \mathbb{K}^n, \quad \mu \in \mathbb{K}, \right. \\ \left. O = \text{Enc}_{\mathcal{C}}(\text{Pack}(\pi_2(\mathbf{b}))), \quad \langle \mathbf{a}, \mathbf{b}_{\mathbb{K}} \rangle_{\mathbb{K}} = \mu \right\}. \quad (23)$$

Direct reductions.

• Reduction 1: Run the linear-relation IOP.

$$\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}) \xrightarrow{\Pi_{\text{iopp}}} \mathcal{R}_{\text{triv}}.$$

- **Proof:** π_{iopp} proves the derived $(\mathbf{a}, \mu)$ claim against the public root $\rho = \text{Root}(O)$.
- **Result:** acceptance or rejection; no public relation remains.
- **Scope:** the final solid paper-level edge. The profile below gives one concrete implementation.

2.7.1 Current profile's internal expansion of Π_{iopp}

The chunked recursive-opening profile implements Π_{iopp} on the MLE-shaped LIN family emitted by Π_{red} as follows. These are internal claim transformations, not additional solid edges in the paper's relation graph.

1. **Per-claim ring switch.** For each active ℓ , the prover transmits

$$s^{(\ell)} = (s_v^{(\ell)})_{v < 128}, \quad s_v^{(\ell)} := \widetilde{B_{\text{bit}, v}}(r_{\text{hi}}^{(\ell)}).$$

After all such vectors are absorbed, the verifier squeezes one shared r'' and derives a packed claim

$$O = \text{Enc}_{\mathcal{C}}(P), \quad \langle B_\ell, P \rangle_{\mathbb{K}} = \beta_\ell. \quad (24)$$

The witness is $P = \text{Pack}(\pi_2(\mathbf{b}))$ and $\rho = \text{Root}(O)$ is unchanged. This step has error $\varepsilon_{\text{ring}}^{(\ell)}$ on the admitted MLE-shaped source; it does not specify a general ring switch for arbitrary LIN coefficients.

2. **Batch.** The verifier squeezes fresh $\boldsymbol{\eta} \in \mathbb{K}^{|\mathcal{A}|}$ and derives, without a proof field,

$$B := \sum_{\ell \in \mathcal{A}} \eta_\ell B_\ell, \quad \beta := \sum_{\ell \in \mathcal{A}} \eta_\ell \beta_\ell.$$

The resulting internal claim is $O = \text{Enc}_{\mathcal{C}}(P)$ and $\langle B, P \rangle_{\mathbb{K}} = \beta$, with batching error $\varepsilon_{\text{batch}}$.

3. **Recursive opening.** The prover transmits $\pi_{\text{open}} = \pi_{\text{lig}}$, one profile-defined recursive constrained-code proof containing its internal proximity messages and authenticated openings. The verifier derives all opening challenges and accepts only if the proof verifies against (ρ, B, β) . This completes Π_{iopp} with error $\varepsilon_{\text{open}}$.

The intended paper ring-switch construction is not used as an independent solid edge here: the three operations above are the selected profile's internal implementation of the already named $\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}) \xrightarrow{\Pi_{\text{iopp}}} \mathcal{R}_{\text{triv}}$ edge.

2.8 The virtual Lin-BitsDual extension

For a public $\mathbb{F}_2$ -linear map $T : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^{n'}$ and $h_0 \in \mathbb{F}_2^{n'}$, let $\text{lift}_2 : \mathbb{F}_2 \rightarrow \{0, 1\} \subset \mathbb{Z}$ act coordinatewise and put $\bar{h} = h_0 + T(\pi_2(\mathbf{b}))$. Define

$$\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virt}} := \left\{ ((T, \mathbf{a}_q, \mu, h_0), O; \mathbf{b}) : \mathbf{b} \in \{0, 1\}^n, \quad \mathbf{a}_q \in \mathbb{F}_q^{n'}, \quad \mu \in \mathbb{F}_q, \right. \\ \left. O = \text{Enc}_{\mathcal{C}}(\text{Pack}(\pi_2(\mathbf{b}))), \quad \langle \pi_q(\text{lift}_2(\bar{h})), \mathbf{a}_q \rangle_{\mathbb{F}_q} = \mu \right\}. \quad (25)$$

Direct reductions.

• Conditional virtual reduction.

$$\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virt}} \xrightarrow{\Pi_{\text{partial-core}}} \mathcal{R}_{h\text{-Lin}}^{\text{virt}}.$$

- **Proof:** π_{virtual} carries the chosen partial-core and GKR messages; the virtual table is not transmitted.
 - **Result:** the verifier derives $(\mathbf{a}_{\mathbb{K}}, \mu')$.
 - **Scope:** available only after a virtual profile defines the proof format, transcript, verifier, and error bound.
- Let $h_{0, \mathbb{K}} := \iota_{2 \rightarrow \mathbb{K}}(h_0)$ and let $T_{\mathbb{K}}$ be the scalar extension of T . Only $\mathbf{b}$ is committed. The virtual linear seam is

$$\mathcal{R}_{h\text{-Lin}}^{\text{virt}} := \left\{ ((T, h_{0, \mathbb{K}}, \mathbf{a}_{\mathbb{K}}, \mu'), O; \mathbf{b}) : \mathbf{b} \in \{0, 1\}^n, \quad \mathbf{a}_{\mathbb{K}} \in \mathbb{K}^{n'}, \quad \mu' \in \mathbb{K}, \right. \\ \left. O = \text{Enc}_{\mathcal{C}}(\text{Pack}(\pi_2(\mathbf{b}))), \right. \\ \left. \langle \mathbf{a}_{\mathbb{K}}, h_{0, \mathbb{K}} + T_{\mathbb{K}}(\mathbf{b}_{\mathbb{K}}) \rangle_{\mathbb{K}} = \mu' \right\}. \quad (26)$$

Its public adjoint rewrite transmits nothing:

$$\mathcal{R}_{h\text{-Lin}}^{\text{virt}} \equiv_{\text{Adj}_{T_{\mathbb{K}}^*}} \mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}),$$

$$\mathbf{a}_b := T_{\mathbb{K}}^*(\mathbf{a}_{\mathbb{K}}), \quad \mu_b := \mu' - \langle \mathbf{a}_{\mathbb{K}}, h_{0, \mathbb{K}} \rangle_{\mathbb{K}}.$$

This exact rewrite need not preserve the MLE coefficient shape admitted by the current internal ring switch. A virtual profile must therefore supply a compatible Π_{iopp} adapter or opening; the direct profile rejects the virtual source.

2.9 The complete relation graph and proof object

Let $\mathcal{R}_{\text{triv}} := \{(\perp, \perp; \perp)\}$. The paper-level relation graph is

$$\begin{aligned}
 \mathcal{R}_{\text{PIMSAT}}^{\text{app}} &\xrightarrow{\text{Encode}} \mathcal{R}_{\text{PIMSAT}}^{\text{enc}}, \\
 \mathcal{R}_{\text{PIMSAT}}^{\text{enc}} &\xrightarrow{\Pi_{\text{app},1}} \mathcal{R}_{\text{PESAT}}^{\text{app}} \xrightarrow{\Pi_{\text{app},2}} \mathcal{R}_{\text{Lin-ModFieldBin}}, \\
 \mathcal{R}_{\text{Lin-ModFieldBin}} &\equiv \text{Bitify } \mathcal{R}_{\text{Lin-BitsDual}} \xrightarrow{\Pi_1} \mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2\mathbb{Z}}, \mathcal{C}, \text{Pack}) \\
 \mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2\mathbb{Z}}, \mathcal{C}, \text{Pack}) &\xrightarrow{\Pi_{\text{red}}} \mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}) \xrightarrow{\Pi_{\text{iopp}}} \mathcal{R}_{\text{triv}}.
 \end{aligned} \tag{27}$$

Encode and Bitify add no proof fields; chunking and recursive opening refine the displayed edges. The semantic proof object is

$$\pi_{\text{F2Z}} = \left(\pi_{\text{PIOP}}, \left(\nu^{(\ell)}, u^{(\ell)}, \pi_{\text{red}}^{(\ell)} \right)_{\ell \in \mathcal{A} \text{ increasing}}, (s^{(\ell)})_{\ell \in \mathcal{A} \text{ increasing}}, \pi_{\text{lig}} \right). \tag{28}$$

Its replay schedule is specified in Section 3. Public inputs are external to the proof; other omitted values are derived. The v1 byte profile partitions transmitted fields, in replay order, as

$$\begin{aligned}
 \pi_{\text{wire}} &:= (N_{\text{F2Z}}, H_{\text{F2Z}}), & \text{narglen}(\pi) &:= |N_{\text{F2Z}}|, & \text{hintlen}(\pi) &:= |H_{\text{F2Z}}|, \\
 & & \text{payloadlen}(\pi) &:= |N_{\text{F2Z}}| + |H_{\text{F2Z}}|, & \text{wirelen}(\pi) &:= |\text{HostEncode}_{\text{v1}}(N_{\text{F2Z}}, H_{\text{F2Z}})|.
 \end{aligned} \tag{29}$$

Benchmarks MUST report all four lengths. Section 5.4 fixes their byte meaning, the outer 72-byte container, and the exact record counts and exhaustion rules. With edge errors $\varepsilon_{\text{app},1}$, $\varepsilon_{\text{app},2}$, $\varepsilon_{\text{red}}^{(\ell)}$, $\varepsilon_{\text{ring}}^{(\ell)}$, $\varepsilon_{\text{batch}}$, and $\varepsilon_{\text{open}}$,

$$\varepsilon_{\text{F2Z}} \leq \varepsilon_{\text{app},1} + \varepsilon_{\text{app},2} + \sum_{\ell \in \mathcal{A}} (\varepsilon_{\text{red}}^{(\ell)} + \varepsilon_{\text{ring}}^{(\ell)}) + \varepsilon_{\text{batch}} + \varepsilon_{\text{open}}.$$

3 Prover–verifier interaction

3.1 Transcript channels and schedule

Equation (29) defines the two proof streams; these badges remain normative without color.

Badge	Bytes	Verifier action
N[x]	NARG	Consume a canonical typed record and absorb that same encoding before any dependent squeeze; PoW alone uses the checked-deferred read in Section 5.6.
H[x]	Hint	Consume, never absorb, and check against the bound prefix before the next squeeze.
D[x]	None	Public, verifier-derived, or challenge value; no proof bytes, with the exact framing and squeeze transition of Section 5.6.
N/H[π]	Both	The subordinate ledger fixes each field’s channel and both replay orders.

Object notation $\text{Verify}(\dots, \pi; \text{tr})$ denotes the equivalent $\text{VerifyStream}(\dots; \text{tr}, c_N, c_H)$. It follows the subordinate ledger over the closed profile’s bounded input slices, performs the same checks and events, and succeeds only at both declared cursor endpoints.

Given public data and bound NARG prefix A , a hint h is admissible only if its accepted value is computationally unique:

$$\Pr[\exists h \neq h' : \text{CheckHint}_i(A, h) = \text{CheckHint}_i(A, h') = 1] \leq \text{negl}(\lambda). \tag{30}$$

The check MUST finish before the next Squeeze. Acceptance requires no pending hint and exact exhaustion of both cursors; missing, misordered, or trailing records reject.

Let $\text{Frame}_{\text{v1}}(\tau, \ell, k, i; z)$ be the byte frame of Equation (44). Before the first proof challenge,

$$\text{tr}_0 := \text{Absorb}(\text{Absorb}(\text{tr}_{\text{init}}, \text{Frame}_{\text{v1}}(0x0001, \perp_{32}, \perp_{32}, \perp_{32}; \text{ProfileV1})), \text{Frame}_{\text{v1}}(0x0002, \perp_{32}, \perp_{32}, \perp_{32}; \text{StatementV1})). \tag{31}$$

The payloads in Equation (31) are fixed by Equation (46). Later absorbs are domain-separated by their numeric tag and fixed (ℓ, k, i) scope fields. A squeeze updates tr and yields a derived, non-transmitted value; each populated interaction row denotes one event.

The tables realize Equation (27) in this exact order:

$$G_1; G_2; G_3; \quad \text{for } \ell \in \mathcal{A} \text{ in increasing order : } (G_4^{(\ell)}; G_5^{(\ell)}); \quad G_6; G_7; G_8; G_9; G_{10}. \tag{32}$$

Right ledgers contain post-group claims; *internal* claims and superscript- (ℓ) copies add no paper relation.

Group 1. Commit and bind the application statement

Prover P	Traffic	Verifier V
Construct $\mathbf{b} := \text{bits}(\mathbf{f}),$ $P := \text{Pack}(\pi_2(\mathbf{b})),$ $O := \text{Enc}_{\mathcal{C}}(P), \quad \rho := \text{Root}(O).$	$\xrightarrow{D[\text{com}]}$ Validate the profile, statement, shape, code, and root type; then set $\text{tr} \leftarrow \text{AbsorbStatement}(\text{tr}, (\text{pp}, x_{\text{app}}, \text{com})).$	REDUCTION LEDGER INITIALIZATION Encode FROM $\mathcal{R}_{\text{PIMSAT}}^{\text{app}}$ TO $\mathcal{R}_{\text{PIMSAT}}^{\text{enc}}$ CURRENT STATE PIMSAT RELATION NEW $p_j^{(a)}(\mathbf{x}, \mathbf{f}) \in I_a$ $(\forall a, j)$ WHERE $\rho \quad \rho = \text{Root}(O)$

Group 2. Run the application PIOP

The displayed edge abbreviates the paper's application-PESAT stage followed by its linear reduction; both retain the same encoded oracle.

Prover P	Traffic	Verifier V
Construct $(\text{tr}, \pi_{\text{PIOP}}) \leftarrow \text{PIOP.Prove}(\text{pp}, x_{\text{app}}, f; \text{tr}).$	$\xrightarrow{N/H[\pi_{\text{PIOP}}]}$ Compute $(\text{tr}, v, y, w, w') \leftarrow \text{VerifyPIOP}(x_{\text{app}}, \rho, \pi_{\text{PIOP}}; \text{tr}),$ require $v_{b,c} = \pi_q(w_b)w'_c$ and all public bounds, and bind the derived core instance before its first challenge.	

Group 3. Bitify the weighted linear relation

Prover P	Traffic	Verifier V
Retain $b = \text{bits}(f)$ and its matrix form B_{bit} .		Derive, without a message or challenge, $u_{c,b,j} := v_{b,c} \pi_q(2^j) = \pi_q(w_b 2^j) w'_c.$ This is retyping by semilinearity, not a Booleanity round. Encode CoreSeamV1 by Equation (47) and absorb its tag-0003 frame at scope $(\perp_{32}, \perp_{32}, \perp_{32})$ before the first chunk record.

Group 4. Run chunk ℓ of Π_1

Prover P	Traffic	Verifier V
Before sending anything, compute $u_c^{(\ell)} = \sum_i \omega_i^{(\ell)} B_{\text{bit}}[c, i], \quad \nu_c^{(\ell)} = g^{u_c^{(\ell)}}.$	$\xrightarrow{N[\nu^{(\ell)}]}$ $\xrightarrow{H[u^{(\ell)}]}$ $\xleftarrow{D[\zeta^{(\ell)}]}$	Read and absorb the canonical typed vector $\nu^{(\ell)}$. Before any later squeeze, require $ u^{(\ell)} = 2^s, \quad 0 \leq u_c^{(\ell)} < 2^{127},$ and $g^{u_c^{(\ell)}} = \nu_c^{(\ell)}$ for every c . The order of g and the range bound make this accepted exponent unique. Then set $\zeta^{(\ell)} \leftarrow \text{Squeeze}_{\mathbb{K}}^s(\text{tr}),$ $e_0^{(\ell)} := \widetilde{\nu^{(\ell)}}(\zeta^{(\ell)}).$

Group 5. Run the GKR reduction for chunk ℓ

The PCS-level schedule treats $\Pi_{\text{red}}^{(\ell)}$ as the named GKR subprotocol. Its input is the current PESAT claim, its unchanged commitment $\rho = \text{Root}(O)$, and the transcript state after $\zeta^{(\ell)}$ and $e_0^{(\ell)}$ are derived. Denote this typed input bundle by $\mathcal{I}_{\text{GKR}}^{(\ell)}$. Its output must have the MLE-shaped coefficient $a_\ell[z] = \text{eq}(z, pt_\ell)$ and satisfy $\widetilde{B_{\text{bit}, \mathbb{K}}}(pt_\ell) = \mu_\ell$ against the same root.

Prover P	Traffic	Verifier V
Construct the selected GKR profile's canonical proof fragment $\pi_{\text{red}}^{(\ell)} \leftarrow \text{GKR.Prove}(\text{pp}_{\text{GKR}}, \mathcal{I}_{\text{GKR}}^{(\ell)}, B_{\text{bit}}; \text{tr}).$	$\xrightarrow{N/H[\pi_{\text{red}}^{(\ell)}]}$	Compute $(\text{tr}', pt_\ell, \mu_\ell) \leftarrow \text{GKR.Verify}(\text{pp}_{\text{GKR}}, \mathcal{I}_{\text{GKR}}^{(\ell)}, \pi_{\text{red}}^{(\ell)}; \text{tr}),$ require success and complete consumption of the GKR fragment, then set $\text{tr} \leftarrow \text{tr}'.$

Retain the witness value $\widetilde{B_{\text{bit}, \mathbb{K}}}(pt_\ell)$.

All internal GKR messages and challenges are owned by the selected GKR profile and use its versioned domain namespace. Treating GKR atomically here does not permit an implementation to omit that subordinate transcript and encoding specification.

For a conforming merged-forest subordinate profile, the repeated phase- x value $H[S_{x,k}^{(\ell)}]$ is a valid hint because the prefix already fixes $e_k^{(\ell)}$ and the verifier immediately requires $S_{x,k}^{(\ell)} = e_k^{(\ell)}$ before any following squeeze. A canonical profile SHOULD omit that redundant field and mark it $D[e_k^{(\ell)}]$ instead. The phase- c claimed sums, sumcheck messages, and terminal child pairs remain $N[\text{GKR messages}]$: their consistency checks do not uniquely determine them and later challenges must bind them.

The outer encoded field `presums[ $\ell$ ].claimed_sums` is distinct from those phase- c claims. Wire profile

REDUCTION LEDGER

RELATION REDUCTION Π_{PIOP}
FROM $\mathcal{R}_{\text{FIMSAT}}^{\text{enc}}$
TO $\mathcal{R}_{\text{Lin-ModFieldBin}}$

CURRENT STATE

LMFB **RELATION** **NEW**
 $\langle v, \psi(f) \rangle_R = y$

WHERE

$\rho \quad \rho = \text{Root}(O)$

REDUCTION LEDGER

DETERMINISTIC REWRITE Bitify
FROM $\mathcal{R}_{\text{Lin-ModFieldBin}}$
TO $\mathcal{R}_{\text{Lin-BitsDual}}$

CURRENT STATE

LBD **RELATION** **NEW**
 $\langle u, \iota_R(b) \rangle_R = y$

WHERE

$\rho \quad \rho = \text{Root}(O)$

REDUCTION LEDGER

PAPER-EDGE FRAGMENT $\Pi_1^{(\ell)}$
FROM $\mathcal{R}_{\text{Lin-BitsDual}}$
TO $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_{2^s}}^{(\ell)}, C, \text{Pack})$

CURRENT STATE

LBD **RELATION** **USED**
 $\langle u, \iota_R(b) \rangle_R = y$
PESAT **RELATION** **NEW**
 $Q_c^{(\ell)}(B_{\text{bit}}) = 0 \ (\forall c)$

WHERE

$\rho \quad \rho = \text{Root}(O)$
 $P \quad P = \text{Pack}(\pi_2(b))$

REDUCTION LEDGER

GKR SUBPROTOCOL $\Pi_{\text{red}}^{(\ell)}$
FROM $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_{2^s}}^{(\ell)}, C, \text{Pack})$
TO $\mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, C, \text{Pack})$

CURRENT STATE

LBD **RELATION**
 $\langle u, \iota_R(b) \rangle_R = y$
LIN **Pack** **RELATION** **NEW**
 $\widetilde{B_{\text{bit}, \mathbb{K}}}(pt_r) = \mu_r$
 $(r \in \mathcal{A}, r \leq \ell)$

WHERE

$\rho \quad \rho = \text{Root}(O)$
 $P \quad P = \text{Pack}(\pi_2(b))$

v1 omits it and supplies the derived value $D[e_d^{(\ell)} + 1]$ to the MultiDegree pre-sumcheck. A decoder that encounters the reserved legacy tag 3081 MUST reject.

Group 6. Complete Π_1 's reconstruction check

Prover P	Traffic	Verifier V
		After all active chunks, set inactive $u^{(\ell)} := 0$, reconstruct $\mu_c^{\text{col}} := \sum_{\ell=0}^{L-1} 2^{c w_\ell} u_c^{(\ell)} \in \mathbb{Z},$ and require the paper's tensor reconstruction equality $\sum_c w'_c \pi_q(\mu_c^{\text{col}}) = y.$ This check has no message, absorb, or challenge.

Group 7. Apply the virtual adjoint when needed

The direct profile skips this optional group and carries its current LIN family unchanged. A virtual profile substitutes its specified partial core for Groups G_4 – G_6 , producing $h = h_{0,\mathbb{K}} + T_K(b_K)$ and $\langle a_h, h \rangle_{\mathbb{K}} = \mu_h$ by the proof-carrying conditional edge

$$\mathcal{R}_{\text{Lin-BitsDual}}^{\text{virt}} \xrightarrow{\Pi_{\text{partial-core}}} \mathcal{R}_{h\text{-Lin}}^{\text{virt}}.$$

Only the following adjoint rewrite is zero-message:

$$a_b := T_K^* a_h, \quad \mu_b := \mu_h - \langle a_h, h_{0,\mathbb{K}} \rangle_{\mathbb{K}}, \quad \langle a_b, b_K \rangle_{\mathbb{K}} = \mu_b. \quad (33)$$

No message or challenge is generated. A profile without this predecessor and a compatible ring adapter rejects virtual inputs.

Group 8. Ring-switch each active linear claim

For the direct MLE-structured profile, split $pt_\ell = (r_{\text{lo}}^{(\ell)}, r_{\text{hi}}^{(\ell)})$ with $|r_{\text{lo}}^{(\ell)}| = 7$. After the shared challenge r'' , the successor coefficients are

$$\begin{aligned} \bar{s}_u^{(\ell)} &:= \sum_v [s_v^{(\ell)}]_u \beta_v, & \Phi_{r''}(z) &:= \sum_u \text{eq}(u, r'')[z]_u, \\ \beta_\ell &:= \sum_u \text{eq}(u, r'') \bar{s}_u^{(\ell)}, & B_\ell(Y) &:= \Phi_{r''}(\text{eq}(r_{\text{hi}}^{(\ell)}, Y)). \end{aligned} \quad (34)$$

Prover P	Traffic	Verifier V
For every $\ell \in \mathcal{A}$ and $0 \leq v < 128$, compute $s_v^{(\ell)} := \widetilde{B_{\text{bit},v}}(r_{\text{hi}}^{(\ell)}).$	$\xrightarrow{N[s^{(\ell)}]}$ For each ℓ, validate and absorb the canonical $s^{(\ell)}$ frame, then require $\sum_{v=0}^{127} \text{eq}(v, r_{\text{lo}}^{(\ell)}) s_v^{(\ell)} = \mu_\ell,$ before any squeeze. After the last vector squeeze $r'' \leftarrow \text{Squeeze}_{\mathbb{K}}^T(\text{tr})$. $\xleftarrow{D[r'']}$	With $[z]_u$ denoting coordinate u in the fixed $\mathbb{K}/\mathbb{F}_2$ basis, derive (B_ℓ, β_ℓ) by Equation (34). Independently derive the same $(B_\ell, \beta_\ell)_{\ell \in \mathcal{A}}$.

Group 9. Batch the packed linear relations

Prover P	Traffic	Verifier V
Form $B := \sum_{\ell \in \mathcal{A}} \eta_\ell B_\ell, \quad \beta := \sum_{\ell \in \mathcal{A}} \eta_\ell \beta_\ell.$	$\xleftarrow{D[\eta]}$	For each active ℓ in increasing order, squeeze $\eta_\ell \in \mathbb{K}$ under tag 4102 and scope $(\ell, \perp_{32}, \perp_{32})$. Independently derive the same (B, β).

REDUCTION LEDGER

PAPER COMPOSITION $\Pi_1; \Pi_{\text{red}}$
FROM $\mathcal{R}_{\text{Lin-BitsDual}}$
TO $\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, C, \text{Pack})$

CURRENT STATE

LIN[Pack] **RELATION** **REDUCED**
 $\widetilde{B_{\text{bit},\mathbb{K}}}(pt_\ell) = \mu_\ell$
 $(\ell \in \mathcal{A})$

WHERE

$\rho \quad \rho = \text{Root}(O)$

REDUCTION LEDGER

OPTIONAL ZERO-MESSAGE BRANCH $\text{Adj}_{T_K^*}$
FROM $\mathcal{R}_{h\text{-Lin}}^{\text{virt}}$
TO $\mathcal{R}_{\text{Lin}}(\mathbb{K}, C, \text{Pack})$

CURRENT STATE

LIN[Pack] **RELATION** **USED**
 $\widetilde{B_{\text{bit},\mathbb{K}}}(pt_\ell) = \mu_\ell$
 $(\ell \in \mathcal{A})$

WHERE

$\rho \quad \rho = \text{Root}(O)$

REDUCTION LEDGER

Π_{IOPP} INTERNAL Π_{ring}^A
FROM $\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, C, \text{Pack})$
TO $\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, C, \text{id})$

CURRENT STATE

LIN[id]^A **RELATION** **NEW**
 $\langle B_\ell, P \rangle_{\mathbb{K}} = \beta_\ell$
 $(\ell \in \mathcal{A})$

WHERE

$\rho \quad \rho = \text{Root}(O)$
 $P \quad P = \text{Pack}(\pi_2(b))$

REDUCTION LEDGER

Π_{IOPP} INTERNAL Batch_η
FROM $\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, C, \text{id})$
TO $\mathcal{R}_{\text{Lin}}(\mathbb{K}, C, \text{id})$

CURRENT STATE

LIN[id] **RELATION** **NEW**
 $\langle B, P \rangle_{\mathbb{K}} = \beta$

WHERE

$\rho \quad \rho = \text{Root}(O)$
 $P \quad P = \text{Pack}(\pi_2(b))$

Group 10. Open the batched claim

Prover P	Traffic	Verifier V
Run the profile-selected polynomial opening protocol—Ligerito in the current profile—for the batched claim (ρ, B, β) , producing π_{open} .	$N/H[\pi_{\text{open}}]$	Accept only if π_{open} verifies against (ρ, B, β) . This authenticates $O = \text{Enc}_C(P)$ and discharges the batched claim.

REDUCTION LEDGER

Π_{IOPF} INTERNAL Π_{open}
FROM $\mathcal{R}_{\text{Lin}}(\mathbb{K}, C, \text{id})$
TO $\mathcal{R}_{\text{triv}}$

CURRENT STATE

nothing left to prove

Recursive-opening channel ledger. For the recursive Ligerito profile, the following partition is normative at the field level. The NARG records are replayed in verifier-event order, not Rust structure-field order; hint records are consumed at the opening level where their checks appear.

Class	Fields	Binding and verification deadline
$N[\cdot]$	Initial and recursive roots; sum-check messages; OOD values; <code>final_proof.yr</code> ; query- and fold-grinding nonces.	Each is observed or checked-and-absorbed before a dependent challenge. PoW nonces remain NARG because multiple values may satisfy the predicate; Section 5.7 fixes their tags and schedule.
$H[\text{open}_0]$	<code>initial_proof.opened_rows</code> and <code>initial_proof.merkle_proof</code> .	The prior initial root and sampled query positions bind the rows and path. Merkle verification must succeed before the next sumcheck-introduction challenge.
$H[\text{open}_i]$	Each recursive proof's <code>opened_rows</code> and <code>merkle_proof</code> .	Verify against the already absorbed preceding-level root and sampled queries before the next batching challenge. Consume levels in increasing recursive order.
$H[\text{open}_{\text{fin}}]$	<code>final_proof.opened_rows</code> and <code>final_proof.merkle_proof</code> .	Verify against the preceding root after the final queries are fixed and before the final batching challenge or acceptance.

The authenticated rows are hints because the earlier Merkle root computationally fixes their accepted contents; the authentication nodes only certify that fixed opening. The whole π_{open} MUST NOT be placed on the hint channel. A recursive-opening implementation must reject before any later squeeze on a failed row-length or Merkle check and must exhaust both substreams before returning success. Section 5.8 fixes the byte payload, scope, cardinality, and exact replay position of every field summarized in this table.

The Rust type `FlockCommitHint` is unrelated terminology: it is a prover-local commitment cache/key, not an H_{F2Z} record. Its rows, packed columns, message, and prover data are not proof bytes; only the public commitment/root enters the verifier-facing protocol. Groups G_8 – G_{10} are the concrete recursive-opening profile's internal expansion of the paper edge Π_{IOPF} . The verifier accepts iff every displayed check succeeds and the proof parser consumes exactly the fields specified by these groups.

4 Verifier algorithm

The verifier rejects on any failed gate or helper, malformed or unsupported value, or cursor mismatch. NARG and hint cursors are independent; a hint stays pending until its predicate succeeds, and every squeeze requires `NoPendingHint`. Records and groups are processed in increasing order. The labels below remain meaningful without color.

[N] read / bind	[H] read / check	[S] challenge	[C] reject gate	[D] local value	[CALL] subprotocol
G1–G3 statement	G4–G5 per chunk	G6–G7 reconstruction	G8–G10 opening		

Procedure VerifyF2Z.

Require: $\text{pp}, x_{\text{app}}, \text{com} = (\rho), \pi_{\text{host}}$

Ensure: true or false

Group 1: Validate and bind the commitment

$$\mathcal{R}_{\text{PIMSAT}}^{\text{app}} \xrightarrow{\text{Encode}} \mathcal{R}_{\text{PIMSAT}}^{\text{enc}}$$

- 1: $(N_{\text{F2Z}}, H_{\text{F2Z}}, n_N, n_H, b_N, b_H) \leftarrow \text{DecodeHostV1}(\pi_{\text{host}})$
- 2: **[C] Require** $[\text{HeaderV1} = 1 \wedge \text{profile_id} = \text{ASCII}(\text{F2ZPCS-v1-B3-SHA256-CORE-0000001})]$
- 3: **[C] Require** $[\text{ValidatePublic}(\text{pp}, x_{\text{app}}, \rho) = 1]$
- 4: **[C] Require** $[\text{ResolveAppProfile} \wedge \text{ResolveGKRProfile} \wedge \text{ResolveCodeProfile}]$
- 5: Read (t, s, W, g) from x_{app} ; **[C] Require** $[t, s \in \mathbb{N} \wedge W \in 2^{\mathbb{N}}]$
- 6: **[D]** $w \leftarrow \log_2 W$, $d \leftarrow t + w$, $m \leftarrow d + s$, $m_p \leftarrow m - 7$, $c_w \leftarrow 127 - t - W$
- 7: **[C] Require** $[d \geq 7 \wedge t + W \leq 126 \wedge 22 \leq m \leq 35 \wedge c_w > 0]$
- 8: **[C] Require** $[\text{ord}(g) = 2^{128} - 1]$
- 9: **[C] Require** $[\theta_2, \theta_3 \in \mathbb{K} \setminus \{0, 1\} \wedge \theta_2 \neq \theta_3]$
- 10: $\text{tr} \leftarrow \text{[N] Absorb}(\text{tr}_{\text{init}}, \text{Frame}_{\text{v1}}(0x0001, \perp_{32}, \perp_{32}, \perp_{32}; \text{ProfileV1}))$ using Equation (46)
- 11: $\text{tr} \leftarrow \text{[N] Absorb}(\text{tr}, \text{Frame}_{\text{v1}}(0x0002, \perp_{32}, \perp_{32}, \perp_{32}; \text{StatementV1}))$ using Equation (46)
- 12: Initialize independent cursors **[D]** $c_N \leftarrow \text{Cursor}(N_{\text{F2Z}}, n_N, b_N)$, $c_H \leftarrow \text{Cursor}(H_{\text{F2Z}}, n_H, b_H)$, and $\text{PendingHint} \leftarrow 0$
- 13: Load the application PIOP and GKR subordinate event ledgers and the canonical constrained-code construction selected by `code_profile_id`

Group 2: Verify the application PIOP

$$\mathcal{R}_{\text{PIMSAT}}^{\text{enc}} \xrightarrow{\Pi_{\text{app},1}} \mathcal{R}_{\text{PESAT}}^{\text{app}} \xrightarrow{\Pi_{\text{app},2}} \mathcal{R}_{\text{Lin-ModFieldBin}}$$

- 14: $(\text{tr}, \mathcal{I}_{\text{LMFB}}, w, w', y) \leftarrow \text{[CALL] VerifyPIOPStream}(\text{pp}, x_{\text{app}}, \rho; \text{tr}, c_N, c_H)$
- 15: **[C] Require** $[|w| = 2^t \wedge |w'| = 2^s \wedge 0 \leq w_b < 2^{100} \wedge w'_c, y \in \mathbb{F}_q]$

Group 3: Apply deterministic Bitify

$$\mathcal{R}_{\text{Lin-ModFieldBin}} \xrightarrow{\text{Bitify}} \mathcal{R}_{\text{Lin-BitsDual}}$$

- 16: **[D]** $\mathcal{I}_{\text{LBD}} \leftarrow \text{Bitify}(\mathcal{I}_{\text{LMFB}})$
- 17: **[C] Require** $[\text{WeightShape}(\mathcal{I}_{\text{LBD}}, w, w', y) = 1]$
- $u_{c,b,j} = \pi_q(w_b 2^j) w'_c$
- 18: **[D]** $x_{\text{core}} \leftarrow ((t, s, W), (w, w', y), g)$
- 19: $\text{tr} \leftarrow \text{[N] Absorb}(\text{tr}, \text{Frame}_{\text{v1}}(0x0003, \perp_{32}, \perp_{32}, \perp_{32}; \text{CoreSeamV1}))$ using Equation (47)
- 20: **[D]** $L \leftarrow \max(\lceil 100/c_w \rceil, 1)$
- 21: **[D]** $\omega_b^{(\ell)} \leftarrow \lfloor w_b / 2^{c_w \ell} \rfloor \bmod 2^{c_w}$
- 22: **[D]** $\mathcal{A} \leftarrow \{\ell < L : \exists b, \omega_b^{(\ell)} \neq 0\}$; **[C] Require** $[\mathcal{A} \neq \emptyset]$
- 23: Set $u^{(\ell)} \leftarrow 0^{2^s}$ for $\ell \notin \mathcal{A}$
- 24: Fix, without advancing either cursor, the remaining semantic schedule $((\nu^{(\ell)}, u^{(\ell)}, \pi_{\text{red}}^{(\ell)})_{\ell \in \mathcal{A} \uparrow}, (s^{(\ell)})_{\ell \in \mathcal{A} \uparrow}, \pi_{\text{open}})$
- 25: **for** $\ell \in \mathcal{A}$ in increasing order **do**

Group 4: Verify chunk ℓ of Π_1

$$\mathcal{R}_{\text{Lin-BitsDual}} \xrightarrow{\Pi_1^{(\ell)}} \mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_{2^2}}^{(\ell)}, C, \text{Pack})$$

- 26: $(\nu^{(\ell)}, F_\nu) \leftarrow \text{[N] ReadNarg}(c_N, 0x2001, \ell, \perp_{32}, \perp_{32})$

```

27: tr ← [N] Absorb(tr, Fν)
28: PendingHint ← 1, (u(ℓ), Fu) ← [H] ReadHint(cH, 0x2081, ℓ, ⊥32, ⊥32)
29: [C] Require[|u(ℓ)| = 2s ∧ 0 ≤ uc(ℓ) < 2127 ∧ (guc(ℓ))c = ν(ℓ)]
30: PendingHint ← 0
31: [D] ωi(ℓ) ← ωb(ℓ) 2j, yi(ℓ) ← gωi(ℓ), where i = (b ≪ w) | j
32: ζ(ℓ) ← [S] Squeezeℝ, 0x2101s(tr; ℓ), e0(ℓ) ← ν(ℓ)(ζ(ℓ))

```

Group 5: Verify the GKR reduction

$$\mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q_{\mathbb{F}_2}^{(\ell)}, \mathcal{C}, \text{Pack}) \xrightarrow[\text{red}]{\Pi^{(\ell)}} \mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, \mathcal{C}, \text{Pack})$$

```

33: [D] IGKR(ℓ) ← GKRIInput(xcore, ρ, y(ℓ), ν(ℓ), ζ(ℓ), e0(ℓ))
34: (okGKR, tr, ptℓ, μℓ) ← [CALL] GKR.VerifyStream(ppGKR, TGKR(ℓ); tr, cN, cH)
35: [C] Require[okGKR = 1]

```

Group 6: Complete the Π₁ reconstruction

```

37: [D] μc ← ∑ℓ=0L-1 2cwℓ uc(ℓ) over ℤ
38: [C] Require[∑c wc πq(μc) = y]
Group 7: Normalize the optional virtual source
Rh-Linvirt ≡ AdjRs RLin(ℝ, C, Pack)

```

```

39: [C] Require[profile = direct]
Virtual profiles use a specified partial-core, adjoint, and compatible ring adapter.

```

Group 8: Verify the ring switch

internal expansion of Π_{lopp}

```

40: for ℓ ∈ A in increasing order do
41:   Write ptℓ = (rlo(ℓ), rhi(ℓ))
42:   (s(ℓ), Fs) ← [N] ReadNarg(cN, 0x4001, ℓ, ⊥32, ⊥32)
43:   [C] Require[|rlo(ℓ)| = 7 ∧ |s(ℓ)| = 128]
44:   tr ← [N] Absorb(tr, Fs)
45:   [C] Require[∑v=0127 eq(v, rlo(ℓ)) sv(ℓ) = μℓ]
46: end for
47: r'' ← [S] Squeezeℝ, 0x41017(tr)
48: for ℓ ∈ A in increasing order do
49:   [D] su(ℓ) ← ∑v [sv(ℓ)]u βv, Φr''(z) ← ∑u eq(u, r'') [z]u
50:   [D] βℓ ← ∑u eq(u, r'') su(ℓ), Bℓ(Y) ← Φr''(eq(rhi(ℓ), Y))
51: end for

```

Group 9: Batch the packed LIN claims

internal expansion of Π_{lopp}

```

52: for ℓ ∈ A in increasing order do
53:   ηℓ ← [S] Squeezeℝ, 0x4102(tr; ℓ, ⊥32, ⊥32)
54: end for
55: [D] B ← ∑ℓ ∈ A ηℓ Bℓ, β ← ∑ℓ ∈ A ηℓ βℓ

```

Group 10: Open the batched claim

profile-selected polynomial opening (Ligerito in the current profile)

```

56: [C] Require[B = DerivedCoefficient(xapp, tr) ∧ dom(B) = {0, 1}mp]
57: tr ← [N] Absorb(tr, Framev1(0x5001, ⊥32, ⊥32, ⊥32; LE32(mp) || Encℝ(β)))
58: [C] Require[[CALL] RecVerifyStream(pp, ρ, B, β; tr, cN, cH) = 1]
59: [C] Require[NoPendingHint = 1 ∧ NargEOF(cN) = 1 ∧ HintEOF(cH) = 1 ∧ HostEOF(πhost) = 1 ∧ Counts(cN, cH) = (nN, nH)]
60: return true

```

The helper verifiers own the checks, absorptions, and challenges specified in Section 3; they do not introduce additional paper-level relation nodes. They also own their subordinate channel ledgers: every NARG read advances only c_N, every hint read advances only c_H, and a helper MUST NOT return success with an unchecked hint. Group 4's N[ν^(ℓ)], H[u^(ℓ)] order is realized by tags 2001 and 2081. Every read, absorb, squeeze, PoW check, and EOF condition uses the byte rules in Section 5; no helper may fall back to structure-field order or an opaque serialization.

5 Wire and transcript profile v1

Normative v1 profile. This section fixes the bytes consumed by a conforming prover and verifier. The profile has two physical proof streams, prefix-free typed records, a BLAKE3 transcript, SHA-256 Merkle roots, and the recursive Ligerito and proof-of-work parameters below. A proof using the legacy positional/bincode codec is not a v1 proof. End-to-end use additionally requires the three statement-bound subordinate identifiers to resolve as specified below.

5.1 Profile identity and fixed algorithms

The eight-byte container magic is the ASCII string F2ZPCS\0\0, the wire version is one, and the 32-byte profile identifier is

$$\text{profile_id} = \text{ASCII}(\text{F2ZPCS-v1-B3-SHA256-CORE-0000001}). \quad (35)$$

The literal in Equation (35) has exactly 32 bytes and hex encoding 46325a5043532d76312d42332d5348413235362d434f52452d3030303030303030. Changing any normative rule in this section requires a new profile identifier.

The transcript and its XOF are unkeyed BLAKE3; Merkle nodes and roots use SHA-256; PoW uses unkeyed BLAKE3-256. The field basis is the polynomial basis of Equation (2). An application PIOP, a GKR implementation, and the constrained-code construction each supply a distinct 32-byte subordinate profile identifier. The public statement binds all three identifiers. An unknown identifier, an application/GKR profile without a byte-exact event ledger, or a code profile without a canonical codeword construction and root vectors is unsupported and MUST be rejected.

For the deployed range 22 ≤ m ≤ 35, let J_m be the number of recursive opening levels. The level shapes and all parameters except the base fold difficulties are fixed by

$$\begin{aligned}
 h_i &:= m - 11 - 3i, & J_m &:= \min\{J \geq 1 : h_{J-1} \in \{3, 4, 5\}\}, \\
 r_i &:= 1 + i, & k_i &:= \begin{cases} 4, & i = 0, \\ 3, & i > 0, \end{cases} \\
 Q_i &:= (183, 90, 60, 45, 36, 31, 27, 24)_i, & g_i &:= 16, \\
 o_i &:= \begin{cases} 0, & i = 0, \\ 1, & i > 0. \end{cases}
 \end{aligned} \quad (36)$$

Here h_i is the message-column exponent, r_i the inverse-rate exponent, k_i the fold count, Q_i the query count, g_i the query-grinding difficulty, and o_i the OOD sample count. Implementations MUST use the integers in Equation (36), not a floating-point recomputation. The remaining base fold difficulties f_i are the following literal table. Only the first J_m entries of the row for m exist.

m	$(f_0, \dots, f_{J_m-1})$
22	(9, 6, 3)
23	(10, 7, 4, 1)
24	(11, 8, 5, 2)
25	(12, 9, 6, 3)
26	(13, 10, 7, 4, 2)
27	(14, 11, 8, 5, 3)
28	(15, 12, 9, 6, 4)
29	(16, 13, 10, 7, 5, 3)
30	(17, 14, 11, 8, 6, 4)
31	(18, 15, 12, 9, 7, 5)
32	(19, 16, 13, 10, 8, 6, 3)
33	(20, 17, 14, 11, 9, 7, 4)
34	(21, 18, 15, 12, 10, 8, 5)
35	(22, 19, 16, 13, 11, 9, 6, 6)

5.2 Primitive canonical encodings

All unsigned integers are fixed-width little-endian. Decoders MUST NOT reduce, truncate, ignore high bits, or convert to a host-sized integer before checking that the value fits. The following encodings are the only canonical ones.

Type	Bytes	Canonicity rule
$u_8, u_{16}, u_{32}, u_{64}, u_{128}$ $a \in \mathbb{K}$	1, 2, 4, 8, 16 16	Unsigned little-endian of exactly the named width. $\text{LE}_{64}(a_{\text{lo}}) \parallel \text{LE}_{64}(a_{\text{hi}})$ in the fixed polynomial basis; coefficient bit j is bit j of the resulting 128-bit word.
$a \in \mathbb{F}_q$	16	$\text{LE}_{128}(\text{can}_q(a))$; reject an integer at least $q = 2^{100} - 15$.
Chunk exponent u	16	$\text{LE}_{128}(u)$; require $u < 2^{127}$.
Nonce	8	$\text{LE}_{64}(n)$.
Merkle value	32	Raw SHA-256 digest bytes, in digest output order.
Vector $[x_0, \dots, x_{n-1}]$	4 plus elements	$\text{LE}_{32}(n)$ followed by canonical elements in index order; n must equal the profile-derived expected count.
Opened rows and path	nested	Vector of rows, each a vector of $\mathbb{K}$ elements, followed by a vector of 32-byte Merkle values. Expected row, width, and path counts are derived before allocation.

Every composite is concatenation without padding. Options, Rust enums, **usize**, varints, serde defaults, and bincode layouts are not wire types. For every accepted primitive or composite x , decoding its encoding must consume the whole bounded slice and re-encoding the result must reproduce the same bytes.

5.3 Merkle rows and canonical multiproofs

For opening level $0 \leq k < J_m$, define the message dimension, tree depth, leaf count, and row width by

$$n_k := h_k + k_k, \quad D_k := h_k + r_k, \quad \Lambda_k := 2^{D_k}, \quad I_k := 2^{k_k}. \quad (37)$$

Thus $n_0 = m - 7$, $n_k = h_{k-1}$ for $k > 0$, $D_k = m - 10 - 2k$, $I_0 = 16$, and $I_k = 8$ for $k > 0$. Let $C_k \in \mathbb{K}^{\Lambda_k \times I_k}$ be the level- k codeword matrix, with leaves and lanes in increasing integer order. Its exact row and tree bytes are

$$\begin{aligned} \text{RowBytes}_k(q) &:= \text{Enc}_{\mathbb{K}}(C_k[q, 0]) \parallel \dots \parallel \text{Enc}_{\mathbb{K}}(C_k[q, I_k - 1]), \\ H_{k,0,q} &:= \text{SHA256}(\text{RowBytes}_k(q)), \\ H_{k,d+1,j} &:= \text{SHA256}(H_{k,d,2j} \parallel H_{k,d,2j+1}), \\ \text{root}_k &:= H_{k,D_k,0}. \end{aligned} \quad (38)$$

Here $\text{Enc}_{\mathbb{K}}$ is the 16-byte encoding in Section 5.2. Equation (38) uses no native-memory cast, row-length prefix, row index, level tag, padding, or leaf/internal-node domain byte. Adding such a byte changes every root and requires a new profile identifier. The constrained-code profile defines how C_k is generated; this section fixes its portable row and tree encoding.

Let the already-derived query set be $A_0 = (q_0 < \dots < q_{Q_0-1})$. At depth d , scan the active positions A_d in increasing order. If both siblings are active, combine the pair once and emit no digest. Otherwise emit the digest at position $p \text{ xor } 1$, using p 's parity to place it on the left or right. Then set $A_{d+1} := \text{sortuniq}\{[p/2] : p \in A_d\}$. The canonical multiproof is the concatenation of emitted digests, bottom-up and in that scan order, and its exact count is

$$M_k := \sum_{d=0}^{D_k-1} |\{p \in A_d : (p \text{ xor } 1) \notin A_d\}|. \quad (39)$$

The level- k opening payload is exactly

$$\text{LE}_{32}(Q_k) \parallel \prod_{j=0}^{Q_k-1} (\text{LE}_{32}(I_k) \parallel \text{RowBytes}_k(q_j)) \parallel \text{LE}_{32}(M_k) \parallel \prod_{u=0}^{M_k-1} \text{path}_k[u], \quad (40)$$

with byte length

$$8 + Q_k(4 + 16I_k) + 32M_k. \quad (41)$$

The verifier derives A_d , M_k , and Equation (41) from the sampled queries before allocating or reading the bounded Hint record, requires rows in q_j order, reconstructs Equation (38), and rejects unused path digests or payload bytes.

5.4 Outer two-stream container

The 72-byte header is fixed as follows. Offsets and sizes are decimal; all multibyte integers use Section 5.2.

offset	size	field
0	8	magic = 46 32 5a 50 43 53 00 00
8	2	wire_version = 01 00
10	2	header_len = 48 00
12	4	flags = 00 00 00 00
16	32	profile_id
48	4	narg_record_count
52	4	hint_record_count
56	8	narg_byte_len

(72)

64	8	hint_byte_len
72	...	NARG records, then Hint records

Writing Hdr_{72} for those exact header bytes, the only v1 host encoding is

$$\text{HostEncode}_{v1}(N, H) := \text{Hdr}_{72} \parallel N \parallel H, \quad \text{bytelen}(\text{HostEncode}_{v1}(N, H)) = 72 + \text{bytelen}(N) + \text{bytelen}(H). \quad (42)$$

Equation (42) implies the four benchmark metrics

$$\begin{aligned} \text{narglen}(\pi) &= \text{bytelen}(N), & \text{hintlen}(\pi) &= \text{bytelen}(H), \\ \text{payloadlen}(\pi) &= \text{bytelen}(N) + \text{bytelen}(H), & \text{wirelen}(\pi) &= 72 + \text{payloadlen}(\pi). \end{aligned} \quad (43)$$

The decoder MUST verify Equation (43) with checked arithmetic before allocating, enforce statement-derived upper bounds, and require input length exactly $72 + \text{narg_byte_len} + \text{hint_byte_len}$. The two declared record counts must match the records actually consumed. Header count and length fields are transport metadata and are not transcript input.

5.5 Typed event frames and registry

Every NARG record, Hint record, public absorb, and squeeze event uses the same prefix-free frame header:

offset	size	field	
0	2	event_tag	u16-le
2	2	reserved = 0	u16-le
4	4	chunk	u32-le
8	4	level	u32-le
12	4	round_or_index	u32-le
16	8	payload_len	u64-le
24	...	payload[payload_len]	

Let $\perp_{32} := 0\text{xffffffff}$ denote an inapplicable scope coordinate. For tag τ , scope (ℓ, k, i) , and payload z , the frame is exactly

$$\text{Frame}_{v1}(\tau, \ell, k, i; z) := \text{LE}_{16}(\tau) \parallel \text{LE}_{16}(0) \parallel \text{LE}_{32}(\ell) \parallel \text{LE}_{32}(k) \parallel \text{LE}_{32}(i) \parallel \text{LE}_{64}(\text{bytelen}(z)) \parallel z. \quad (44)$$

Thus Equation (44) has length $24 + \text{bytelen}(z)$. A stream record with a reserved nonzero word, an inapplicable coordinate other than $\perp_{32}$, a wrong channel, a wrong expected scope, or an unknown tag is malformed.

The base-tag registry is below. D events are public or derived, have no proof record, and are absorbed at their ledger position; N records occur only in N and are absorbed; H records occur only in H and are checked but never absorbed; S tags are squeeze requests. For an S base tag τ , the squeeze-result tag is τ OR 0x8000 and is derived, never transmitted. For an S row, the last column describes the derived result; its request payload is always $\text{LE}_{32}(n) \parallel a$ as fixed by Equation (48).

Tag	Class	Event	Scope; proof payload or squeeze result
0001	D	profile/domain	$(\perp, \perp, \perp)$; ProfileV1 from Equation (46).
0002	D	statement	$(\perp, \perp, \perp)$; StatementV1 from Equation (46).
0003	D	core seam	$(\perp, \perp, \perp)$; CoreSeamV1 from Equation (47).
1000-1fff	profile	application PIOP	The selected application profile owns a disjoint, byte-exact ledger within this range.
2001	N	chunk ν	$(\ell, \perp, \perp)$; vector of 2^s elements of $\mathbb{K}$.
2081	H	chunk u	$(\ell, \perp, \perp)$; vector of 2^s canonical $u_{128} < 2^{127}$.
2101	S	chunk ζ	$(\ell, \perp, i)$ for $0 \leq i < s$; one $\mathbb{K}$ challenge.
3001	N	GKR phase-c sum	$(\ell, k, \perp)$; one $\mathbb{K}$ element, absorbed after the phase-link check and before the first phase-c round.
3002-30ff	profile	GKR messages	The selected GKR profile owns exact message tags, codecs, and checks in this range.
3100-31ff	profile	GKR squeezes	The selected GKR profile owns exact squeeze tags and result types in this range.
4001	N	ring s vector	$(\ell, \perp, \perp)$; vector of exactly 128 elements of $\mathbb{K}$.
4101	S	shared r''	$(\perp, \perp, i)$ for $0 \leq i < 7$; one $\mathbb{K}$ challenge.
4102	S	batch η	$(\ell, \perp, \perp)$ in increasing active-chunk order; one $\mathbb{K}$ challenge.
5001	D	opening target	$(\perp, \perp, \perp)$; $\text{LE}_{32}(m_p) \parallel \text{Enc}_{\mathbb{K}}(\beta)$. B is recomputed.
5002	N	initial root	$(\perp, 0, \perp)$; one 32-byte SHA-256 digest.
5003	N	recursive root	$(\perp, k, \perp)$, $1 \leq k < J_m$; one digest.
5004	N	sumcheck message	scope by kind below; kind : u_s followed by $\text{Enc}_{\mathbb{K}}(u_0) \parallel \text{Enc}_{\mathbb{K}}(u_2)$.
5005	N	OOD value	$(a, k, \perp)$; one $\mathbb{K}$ element for sample a .
5006	N	final y_r	$(\perp, J_m - 1, \perp)$; vector of exactly $2^{hJ_m - 1}$ elements of $\mathbb{K}$.
5007	N	fold-PoW nonce	$(\perp, k, i)$; one u_{64} .
5008	N	query-PoW nonce	$(\perp, k, \perp)$; one u_{64} .
5080	H	initial opening	$(\perp, 0, \perp)$; Equation (40).
5081	H	recursive opening	$(\perp, k, \perp)$, $1 \leq k < J_m - 1$; Equation (40).
5082	H	final opening	$(\perp, J_m - 1, \perp)$; Equation (40).
5101	S	fold challenge	$(\perp, k, i)$; one $\mathbb{K}$ challenge.
5102	S	OOD point	(a, k, c) ; one $\mathbb{K}$ coordinate, with c increasing.
5103	S	glue challenge	$(\perp, k, i)$ as fixed below; one $\mathbb{K}$ challenge.
5104	S	PoW seed	$(\perp, k, i)$; 16 bytes; request auxiliary payload C .
5105	S	query draw	$(\perp, k, a)$; one $\mathbb{K}$ challenge for attempt a .
5106	S	query alpha	$(\perp, k, c)$; one $\mathbb{K}$ challenge.

The application and GKR ranges are black-box namespaces, not permission to use implementation-defined bytes. Their profile identifiers must select published ledgers with the same columns as this table. The fixed 3001 event resolves the PCS-level phase-c placement ambiguity in Gap G1; the rest of that gap remains subordinate-profile work and no profile may reassign the tag. The v1 outer pre-sumcheck omits the redundant $\text{presums}[\text{ell}].\text{claimed_sums}$ field and derives $[e_d^{(\ell)} + 1]$, so tag 3081 is reserved and MUST NOT occur.

5.6 Transcript transitions and replay

Model the transcript state as the byte string hashed by an unkeyed BLAKE3 hasher. It starts empty. The profile frame 0001 and statement frame 0002 are absorbed, in that order, before any proof-dependent squeeze. The application statement digest is

$$d_{\text{app}} := \text{BLAKE3}_{256}(\text{hex}^{-1}(46325a2d5043532f6170702d73746174656d656e742f763100) \parallel \text{app_profile_id} \parallel \text{app_statement_bytes}). \quad (45)$$

The hex prefix in Equation (45) is the ASCII literal F2Z-PCS/app-statement/v1 followed by one NUL byte. The two initial frame payloads are exactly

$$\begin{aligned} \text{ProfileV1} &:= \text{hex}^{-1}(46325a5043530000) \parallel \text{LE}_{16}(1) \parallel \text{profile_id}, \\ \text{StatementV1} &:= \text{app_profile_id}[32] \parallel \text{gkr_profile_id}[32] \parallel \text{code_profile_id}[32] \parallel d_{\text{app}}[32] \parallel \text{LE}_{32}(t) \parallel \text{LE}_{32}(s) \parallel \text{LE}_{32}(W) \\ &\quad \parallel \text{Enc}_{\mathbb{K}}(g) \parallel \rho[32]. \end{aligned} \quad (46)$$

Thus ProfileV1 and StatementV1 have 42 and 188 bytes, respectively. The application profile must make its final byte string in Equation (45) canonical.

The derived core-seam payload is

$$\text{CoreSeamV1} := \text{LE}_{32}(2^t) \parallel \prod_{b=0}^{2^t-1} \text{LE}_{128}(w_b) \parallel \text{LE}_{32}(2^s) \parallel \prod_{c=0}^{2^s-1} \text{LE}_{128}(\text{can}_q(w'_c)) \parallel \text{LE}_{128}(\text{can}_q(y)), \quad (47)$$

with exactly $24 + 16(2^t + 2^s)$ bytes. It is absorbed under tag 0003 after deterministic Bitify and before the first tag-2001 record.

For a canonical frame F , absorption appends the exact frame bytes. A squeeze under base tag $\tau < 0x8000$, scope (ℓ, k, i) , output length n , and auxiliary request payload a is the transition

$$\begin{aligned} R &:= \text{Frame}_{v1}(\tau, \ell, k, i; \text{LE}_{32}(n) \parallel a), \\ \text{tr}' &:= \text{tr} \parallel R, \\ z &:= \text{BLAKE3-XOF}(\text{tr}')[0 \dots n], \\ \text{tr}'' &:= \text{tr}' \parallel \text{Frame}_{v1}(\tau \text{ OR } 0x8000, \ell, k, i; z). \end{aligned} \quad (48)$$

Equation (48) uses a clone/finalize-XOF of the incremental hasher after R ; it does not reset or key the hasher. A $\mathbb{K}$ challenge has $n = 16$ and decodes by Section 5.2. A vector challenge is a sequence of scalar squeezes, not one long squeeze. Unless its registry row says otherwise, the scalars use consecutive i values and the auxiliary request payload a is empty. Tag 4102 instead uses $(\ell, \perp, \perp)$ for each active ℓ in increasing order. Tag 5103 uses its one-byte glue kind, and tag 5104 uses the PoW context C ; these are the only nonempty core auxiliary payloads.

On a normal NARG read, the verifier validates the expected tag, scope, payload length, and canonical payload, then immediately appends the complete frame from Equation (44). A PoW nonce is the sole deferred NARG read: the verifier parses its expected canonical frame, permits no intervening event, checks Equation (50), and appends the frame only on success. On a Hint read, it validates the same framing but never appends those bytes. It must complete the hint predicate before the next squeeze. In particular, it reads and absorbs 2001, then reads 2081 and verifies the exponent bounds and $g^u = \nu$ before the first 2101 event. NARG replay follows verifier-event order, never Rust structure-field order. Acceptance requires no deferred record or pending hint and exact logical and byte exhaustion of both streams and every nested slice.

5.7 Grinding and query derivation

Let kind byte zero denote a fold grind and kind byte one a query grind. For kind K_{pow} , level k , round i , and derived difficulty d , define the exact PoW context

$$C := \text{ASCII}(\text{F2Z-PCS/PoW/v1}) \parallel \text{LE}_8(K_{\text{pow}}) \parallel \text{LE}_{32}(k) \parallel \text{LE}_{32}(i) \parallel \text{LE}_{16}(d). \quad (49)$$

In Equation (49), $i = \perp_{32}$ for a query grind. The seed is the 16-byte output of Equation (48) under tag 5104, the same scope, and auxiliary request payload C . After obtaining the seed, the verifier reads the expected 5007 or 5008 NARG frame without absorbing it and requires, for the scheduled positive difficulty d ,

$$\text{LZ}(\text{BLAKE3}_{256}(C \parallel \text{seed}_{16} \parallel \text{LE}_{64}(n))) \geq d, \quad d > 0. \quad (50)$$

LZ in Equation (50) counts zero bits from the most-significant bit of digest byte zero, then proceeds toward the least-significant bit and subsequent bytes. Any satisfying nonce is accepted; minimality is not checked. Only after the predicate succeeds does the verifier absorb the complete nonce frame, before any dependent squeeze. Difficulty zero has no predicate invocation, seed squeeze, or nonce record in this profile.

The exact scheduled difficulties are

$$d_k^{\text{query}} := g_k = 16, \quad d_{k,i}^{\text{fold}} := \max(f_k - i, 0). \quad (51)$$

Equation (51) is evaluated with ordinary nonnegative integers. After the preceding sumcheck message and immediately before fold challenge (k, i) , perform both the tag-5104 seed squeeze and tag-5007 nonce record if $d_{k,i}^{\text{fold}} > 0$; if the difficulty is zero, perform neither event. For $0 \leq k < J_m - 1$, query event k occurs after the tag-5003 commitment root for level $k + 1$, all o_{k+1} OOD bindings for that root, and their glue challenges, and immediately before the tag-5105 draws that open level k . Query event $J_m - 1$ instead occurs after the tag-5006 final y_r , with no new root or OOD binding, and immediately before the final-level draws. There is exactly one query nonce at every level. Consequently the exact number of nonce records is

$$N_{\text{pow}} = J_m + \sum_{k=0}^{J_m-1} \min(k, f_k). \quad (52)$$

Missing, extra, reordered, or trailing nonces violate Equation (52) and reject.

For level k , each 5105 event yields $r \in \mathbb{K}$ and proposes

$$q := r_{10} \bmod 2^{h_k + r_k}. \quad (53)$$

The verifier repeats Equation (53) until Q_k distinct positions exist, discarding duplicates, and then sorts the positions in increasing order. It then derives the tag-5106 alphas before reading and checking the opening.

5.8 Recursive-opening replay ledger

This subsection fixes the complete replay order for the recursive opening. The four canonical tag-5004 message kinds and scopes are

0x00	Start	$(\perp, 0, \perp)$	exactly once,	
0x01	FoldNext	$(\perp, k, j)$	$0 \leq j < k_k$,	
0x02	OodIntro	$(a, k, \perp)$	$1 \leq k < J_m, 0 \leq a < o_k$,	
0x03	QueryIntro	$(\perp, k, \perp)$	$0 \leq k < J_m - 1$.	(54)

Each payload is the one-byte kind followed by the two canonical $\mathbb{K}$ elements in the registry. Any other kind/scope pair rejects.

The outer `VerifyF2Z` caller first requires that B is the deterministic coefficient function derived in Group 9, with domain size 2^{m_p} ; B is never densely serialized. The caller alone absorbs tag 5001 with payload $\text{LE}_{32}(m_p) \parallel \text{Enc}_{\mathbb{K}}(\beta)$ and then invokes `RecVerifyStream`. That helper starts after tag 5001: its first event reads tag 5002, requires the root to equal ρ , absorbs it, and then reads and absorbs the `Start` tag-5004 message.

For each $k = 0, \dots, J_m - 1$ in increasing order, it then performs the following ledger without interleaving another event.

1. For $j = 0, \dots, k_k - 1$, perform the optional fold grind exactly as in Section 5.7, squeeze tag 5101 at $(\perp, k, j)$, then read, validate, and absorb the `FoldNext` tag-5004 message at the same scope.
2. If $k < J_m - 1$, read and absorb the tag-5003 root at $(\perp, k + 1, \perp)$. For each $a = 0, \dots, o_{k+1} - 1$, squeeze the n_{k+1} tag-5102 coordinates at scopes $(a, k + 1, c)$ for $c = 0, \dots, n_{k+1} - 1$; read and absorb tag 5005 at $(a, k + 1, \perp)$; read and absorb the `OodIntro` tag-5004 message at that scope; and squeeze tag 5103 at $(\perp, k + 1, a)$ with the one-byte auxiliary kind 00.
If instead $k = J_m - 1$, read and absorb tag 5006 at $(\perp, k, \perp)$, requiring exactly 2^{h_k} canonical $\mathbb{K}$ elements.
3. Perform the query grind at $(\perp, k, \perp)$ with difficulty 16 and absorb its validated tag-5008 nonce. Starting with attempt $a = 0$, squeeze tag 5105 at $(\perp, k, a)$ and increment a after every squeeze, including a duplicate, until Q_k distinct positions have been obtained. Reject rather than wrap if the next attempt would exceed $2^{32} - 1$. Sort the positions, then squeeze exactly $\lceil \log_2 Q_k \rceil$ tag-5106 alphas at $(\perp, k, c)$ for consecutive c . Read exactly one opening Hint using tag 5080 for $k = 0$, tag 5081 for $0 < k < J_m - 1$, or tag 5082 for $k = J_m - 1$. Check Equations (40)–(41) and the root before any later event. For $k < J_m - 1$, next read and absorb the `QueryIntro` tag-5004 message. At every level, including the final level where no such message exists, squeeze the query glue tag 5103 at $(\perp, k, o_k)$ with one-byte auxiliary kind 01. At the final level this challenge is used directly in the residual check.

The fixed record/event cardinalities are therefore

$$\begin{aligned}
\#5002 &= 1, & \#5003 &= J_m - 1, & \#5005 &= \sum_{k=1}^{J_m-1} o_k, \\
\#5004 &= 1 + \sum_{k=0}^{J_m-1} k_k + \sum_{k=1}^{J_m-1} o_k + (J_m - 1), & \#5006 &= 1, & \#508* &= J_m, \\
\#5101 &= \sum_{k=0}^{J_m-1} k_k, & \#5102 &= \sum_{k=1}^{J_m-1} o_k n_k, & \#5103 &= J_m + \sum_{k=1}^{J_m-1} o_k, \\
\#5106 &= \sum_{k=0}^{J_m-1} \lceil \log_2 Q_k \rceil.
\end{aligned} \tag{55}$$

Here #508* is one opening Hint per level. Tag-5105 count is the transcript-derived number of attempts, all of which appear in the audit log. Nonce counts remain Equation (52). A different order, scope, kind, cardinality, or unread field rejects.

5.9 Parser, implementation, and conformance requirements

A v1 implementation MUST:

1. validate magic, version, flags, profile and subordinate profile identifiers, all profile-, statement-, and transcript-derived bounds, and all reserved fields before allocating or doing proof work;
2. use checked arithmetic and checked conversion for every length and index, reject noncanonical field values, and return a decoding error rather than panic on adversarial bytes;
3. encode the NARG stream in the exact absorb order and the Hint stream in its exact check order, with independent cursors;
4. replace opaque recursive-proof serialization with the field records and exact order in Sections 5.5 and 5.8; and
5. reject unless every outer, subordinate, recursive, row, and Merkle-path slice is exactly exhausted.

Conformance fixtures MUST include canonical primitive and frame hex, one complete $N/H/\text{HostEncode}_{v1}$ proof, the ordered transcript audit log, every squeezed challenge, PoW seeds/nonces and query positions, and all four lengths from Equation (43). Negative fixtures MUST cover an unknown version/profile/tag, a reserved bit, wrong endian or channel, record reordering, truncation and suffix bytes, noncanonical scalars, count overflow, a failed or late u/ν hint, altered PoW scope/difficulty/nonce, and missing or extra recursive fields. Native, recursive, and independent reference verifiers must produce byte-identical transcript audit logs.

The repository file `wire-v1-vectors.txt` is the normative initial known-answer set for the profile identifier, primitive container/frame codec, Merkle row/tree/multiproof codec, and PoW predicate at difficulties 1, 7, 8, and 16. A conforming implementation must reproduce it exactly and add a complete proof/transcript fixture before claiming end-to-end v1 interoperability.

6 Normative gap register

A gap is not permission to infer a rule. Unless the selected profile supplies the closure named below, the verifier MUST reject the affected instance. Rows carrying a closure qualifier distinguish a fixed core rule from remaining subordinate-profile, implementation, fixture, or provenance work.

ID	Source omission or ambiguity	Behavior of this specification	Required closure
G1	The GKR reduction's phase-c claimed sum $S_{c,k}^{(\ell)}$ is transmitted, but the core reduction does not by itself prescribe its transcript treatment. If $\text{eq}(r_{x,k}^{(\ell)}, z_{x,k}^{(\ell)}) = 0$, the phase link does not determine that sum. The current Rust merged-forest verifier uses the claimed sum only after its phase-c challenges and does not absorb it.	The PCS-level schedule treats GKR atomically. The selected GKR profile must absorb the tag-3001 frame fixed by Section 5.5 for $S_{c,k}^{(\ell)}$ after the phase-link check and before the first phase-c round message. Its already-known $S_{x,k}^{(\ell)} = e_k^{(\ell)}$ field is either omitted or carried as a checked, non-absorbed hint.	An interoperable GKR profile must pin its remaining events in the reserved 3002-31ff ranges, and the current verifier must insert the tag-3001 absorb at the stated deadline; alternatively the profile must specify a canonical degenerate-branch rejection rule with its stated error bound.
G2	The paper-level edges are $\Pi_{\text{red}} : \mathcal{R}_{\text{PESAT}}(\mathbb{K}, Q, C, \text{Pack}) \rightarrow \mathcal{R}_{\text{LIN}}(\mathbb{K}, C, \text{Pack})$ and $\Pi_{\text{lopp}} : \mathcal{R}_{\text{LIN}}(\mathbb{K}, C, \text{Pack}) \rightarrow \mathcal{R}_{\text{triv}}$. The concrete path exposes an MLE-specialized LIN claim before changing its representation.	The current profile delegates Π_{red} to the selected GKR reduction, whose contract returns the MLE-shaped LIN claim. Ring switching, batching, and recursive opening are only the internal expansion of Π_{lopp} , with error $\sum_{\ell \in \mathcal{A}} (\varepsilon_{\text{red}}^{(\ell)} + \varepsilon_{\text{ring}}^{(\ell)}) + \varepsilon_{\text{batch}} + \varepsilon_{\text{open}}$.	No runtime closure is missing for the direct profile. A profile claiming the abstract theorem must fix coordinate order and prove both that its concrete Π_{red} realizes the named LIN successor and that its internal opening path realizes Π_{lopp} from that LIN relation to $\mathcal{R}_{\text{triv}}$, including the composed soundness loss.

ID	Source omission or ambiguity	Behavior of this specification	Required closure
G3	A virtual seam may yield the general coefficient $T_{\mathbb{K}}^*(a_{\mathbb{K}})$, whereas the concrete ring switch accepts only its declared LIN source family (MLE-shaped in the core profile).	The direct verifier rejects virtual inputs. A virtual extension may apply the public adjoint without a message only after verifying its own partial-core predecessor, and then rejects unless the resulting $\mathcal{R}_{\text{Lin}}(\mathbb{K}, C, \text{Pack})$ claim is admitted by its internal Π_{Iopp} adapter.	The extension must type T, T^* , specify the virtual partial-core seam, and prove either that every adjoint coefficient lies in the admitted ring-source family or supply a compatible Π_{Iopp} expansion, transcript, verifier, and composed error.
G4	The general relation permits a zero functional, while the active-chunk core has $\mathcal{A} = \emptyset$ and therefore no PESAT grand-product or LIN-to- $\mathcal{R}_{\text{triv}}$ path.	The core profile requires $\mathcal{A} \neq \emptyset$ and rejects otherwise.	A zero-functional profile must still define how ρ is validated and must give its own proof object, transcript, and acceptance predicate.
G5	An application PIOP may output an arbitrary scalar Lin-ModFieldBin coefficient, while the Lin-BitsDual core requires the separable tensor $u_{c,b,j} = \pi_q(w_b 2^j) w'_c$.	The end-to-end verifier accepts only VerifyPIOP outputs whose deterministic bitification equals the declared weighted tensor; the same root must cross the handoff.	The application profile must specify the PIOP proof and transcript handoff plus a weighted-shape adapter, and prove that its Lin-ModFieldBin successor bitifies to either a weighted instance of $\mathcal{R}_{\text{Lin-BitsDual}}$ or its declared weighted virtual Lin-BitsDual source, with a composed soundness bound.
G6 WIRE FIXED	Semantic framing alone does not choose interoperable bytes, the outer two-stream container, a recursive-opening configuration, or a proof-of-work predicate and difficulty. The current one-stream codec writes u but absorbs its derived image g^u , which cannot be represented by the selected two-stream wrapper's write-and-absorb NARG operation.	Section 5 fixes the wire/transcript question: it gives the profile identifier; primitive, vector, record, and outer encodings; numeric event registry; transcript transition; NARG/hint replay; recursive Ligerito integer tables and event ledger; Merkle row and multiproof bytes; PoW context, nonce, predicate, schedule, and difficulty; all four length metrics; and exact nested and outer exhaustion. It sends and absorbs $\nu^{(\ell)} = g^{u^{(\ell)}}$ under tag 2001, then checks the tag-2081 $u^{(\ell)}$ hint before tag 2101. A missing or unknown application/GKR event ledger or constrained-code construction is a hard reject, not an inferred encoding.	Owner: John Wu. Port Section 5 to the host and recursive verifiers, remove the opaque bincode payload, and publish the required complete proof, transcript, Merkle, PoW, and negative fixtures. The current Rust codec remains nonconforming and its legacy bytes are not v1. End-to-end interoperability may be claimed only after the selected application and GKR identifiers resolve to published byte ledgers and the statement-bound code identifier resolves to a complete canonical RS/LCH construction (including domain, basis, padding, and coordinate order) with independent commitment/root vectors. The wire namespaces and PoW rules themselves are not left to implementation inference.
G7 RULE FIXED	The current encoded outer pre-sumcheck includes <code>presums[ℓ].claimed_sums</code> , and current recursive/host decoders may accept unused roots, nonces, recursive proofs, or suffix bytes. The presum is currently checked only after the MultiDegree challenges.	Wire profile v1 omits that redundant field and derives $[e_d^{(\ell)} + 1]$; the reserved legacy tag 3081 rejects. Sections 5.3, 5.5, 5.8, and 5.9 derive the exact counts for roots, nonces, proofs, rows, and query-dependent paths and require logical and byte EOF at every recursive and outer boundary.	Implement the v1 omission and exhaustion checks in both host and recursive verification. Until that implementation work lands, the current Rust path is nonconforming; no alternate v1 field or suffix behavior is permitted.